Comparative Analysis of Network Intrusion Detection Systems for Small Business Environments

A practical evaluation of Snort, Suricata and Zeek under realistic resource constraints

Abdulrahman Rahaq

ABSTRACT

Small businesses face the same kinds of network attacks as larger companies, but they rarely have the budget, staff, or hardware to defend themselves the same way. This project compares three of the most widely used open-source intrusion detection systems, Snort, Suricata and Zeek, inside a VirtualBox lab built to look something like an actual SMB setup, with a single monitoring host running on 2 GB of RAM and two CPU cores. Each tool was tested against four common attacks: a port scan, an ICMP flood, a SYN flood, and an SSH brute force. Snort installed easily and detected the port scan and both floods, but it did not pick up the brute force at all. Suricata would not even start with its default rule set on a 2 GB VM; the OS killed it for using too much memory, and it only ran after the rules were cut down by hand, after which it caught two of the four. Zeek did not produce real time alerts, but its log files captured every SSH attempt clearly, making it the only tool to leave proper forensic evidence of the brute force. A separate test showed that running Snort and Zeek together during a flood pushed the host's RAM to 83% and made SSH stop responding entirely. Overall, the findings suggest no single tool fits every SMB.

INTRODUCTION

Almost every organisation today runs a network connection of some sort, and the volume of attacks targeting those networks has increased steadily for over a decade. An Intrusion Detection System, or IDS, is a piece of software that watches network traffic and flags anything that appears malicious or out of place. It does not actually stop an attack on its own; instead, it raises an alarm so that someone or something else can intervene. For larger organisations with dedicated security teams, the IDS is just one component of a much larger defensive setup. For smaller ones, which describes most small and medium sized businesses, or SMBs, it may be the only network monitoring tool they ever deploy.

Three open-source IDS tools appear repeatedly in both academic literature and the wider security community. Snort was originally written by Martin Roesch in the late 1990s and is generally regarded as the reference implementation of signature-based detection (Roesch, 1999). Suricata emerged roughly a decade later, built from scratch with multithreading in mind to keep pace with faster networks. Zeek, originally called Bro, takes a fundamentally different approach; rather than matching packets against a rule set and generating alerts when a rule fires, it parses each protocol and produces structured logs for later analysis (Paxson, 1999). All three are free, each has an active user community, and all three appear regularly in vendor comparison articles.

What remains far less clear from existing comparisons is which of these tools an SMB should actually choose. Most published studies are conducted on dedicated server class hardware and focus on metrics such as throughput at 10 Gbps, figures that bear little resemblance to the equipment a five person business might possess. The gap this project aims to address is straightforward: while considerable comparative work on Snort, Suricata and Zeek exists, very little of it evaluates these tools under the constraints that small businesses genuinely face: a single low specification server with 2 to 4 GB of memory, no dedicated security analyst, and an IT generalist who likely lacks the time to hand tune a rule set. Furthermore, little has been written about whether running more than one of these tools simultaneously,

which would theoretically provide better coverage, is actually feasible on such modest hardware. Without this evidence, an SMB owner is largely left to choose based on marketing material and forum threads, neither of which serves as a reliable guide.

Aim. To compare the detection capability, resource consumption and overall practicality of Snort, Suricata and Zeek when deployed in an environment that resembles a typical small business network.

To get there, the project has six concrete objectives:

- Build a virtual network in VirtualBox that reflects the kind of hardware and topology a small business would actually use.
- Install and configure each of the three tools on a single monitoring host limited to 2 GB of RAM and two CPU cores.
- Run a fixed set of four attacks – a port scan, an ICMP flood, a SYN flood and an SSH brute force – against the same target while each tool is monitoring.
- Record what each tool detected, what it missed, and how much CPU and memory it used while running.
- Test what happens when more than one tool runs at the same time during an active attack, to see whether multi-tool deployment is realistic on this kind of hardware.
- Use the results to make an evidence-based recommendation for SMBs choosing between signature-based detection (Snort or Suricata) and protocol-based analysis (Zeek).

Those objectives translate directly into four research questions, each of which gets answered with the experimental evidence later on:

- RQ1. Which of the three tools provides the most reliable detection on a host with 2 GB of RAM and no expert tuning?
- RQ2. What configuration barriers does each tool present to an administrator who isn't a security specialist?
- RQ3. Can two or more of these tools run at the same time on commodity hardware during an active attack without breaking something?
- RQ4. What practical trade-offs exist between signature-based real-time alerting and Zeek's log-based approach?

The rest of the report follows the standard structure for a final year project. Section 2 reviews the academic literature on intrusion detection and on the three specific tools, drawing out the gap that this study tries to fill. Section 3 explains the methodology – the virtual network, the attack choices, the data collected, and the ethics around it. Section 4 presents the results from each tool against each attack, with tables and figures. Section 5 discusses what those results mean in relation to the research questions and literature. Section 6 sums up the principal conclusions and gives some practical recommendations.

LITERATURE REVIEW

Introduction

Intrusion Detection Systems (IDS) are one of the most important components of network security as organisations heavily rely on digital infrastructure. This section will review existing research on IDS technologies focusing on detection approaches, performance evaluation methods, and the effectiveness of open-source IDS tools. The reports used will focus specifically on Snort, Suricata and Zeek as these systems are the most used open-source systems both in academic research and industry environments. Examining previous articles will help identify current strengths and limitations of IDS technologies while highlighting gaps in research, especially regarding their suitability for small and medium-sized organisations with limited cybersecurity capabilities.

NIDS and detection approaches

Intrusion Detection Systems are designed to monitor network traffic and detect any malicious or suspicious activity. IDS technologies have been widely adopted in recent times; however, research shows that the detection methods used heavily impact their effectiveness. IDS tools are classified as either signature-based or anomaly-based systems, although under recent developments they are more likely to be combined.

Signature-based IDS tools, such as Snort, rely on set rules or signatures to detect known threats. When Snort was first developed, Roesch (1999) described it as a network intrusion detection platform that was both lightweight and powerful, with the capability to analyze traffic in real time. Over time, Snort became widely used due to its large rule database and strong community support. Studies such as Ghazi et al. (2024) show that when testing known attack datasets, Snort has high detection accuracy, strong precision and recall scores under controlled conditions. Something worth noting about Snort, though, is that it has never really been an application-aware engine in the way some newer tools are. It looks at traffic matching its rules and acts when there's a match – pre-processors handle decoding and decompression, but the engine itself reasons in bytes and patterns, not in terms of what application produced the data. That's part of why it's fast and small, and it's also part of why it sometimes misses things a more application-aware engine could catch.

Sommer and Paxson (2010) mention that attackers frequently alter known attack techniques to attempt to bypass the IDS, which can reduce the effectiveness of signature-based IDS systems. Their research raises concerns regarding whether signature-based detection alone can provide sufficient protection against modern cyber threats. While signature-based IDS tools remain widely used, their reliance on historical attack data suggests that detection capabilities may decline as new attack techniques continue to emerge. Anomaly-based IDS solutions try to overcome these limitations by analysing patterns of network behaviour rather than relying entirely on historical data. Zeek represents a behavioural monitoring system that focuses on identifying deviations from normal network activity (Paxson, 1999). It's worth flagging here that although Zeek is sometimes loosely lumped in with anomaly-based detection, its actual approach is closer to what's called protocol-based or specification-based analysis – it decodes each protocol and writes the result out in structured logs, leaving the question of what's normal up to whoever reads the logs later. Anomaly-based detection techniques more generally offer the potential to detect previously unknown attacks, including zero-day vulnerabilities. However, anomaly-based detection introduces new challenges. Garcia-

Teodoro et al. (2009) found that anomaly-based IDS systems often produce many false alerts compared to signature-based detection. The excessive alert generation may lower the trust the administrators have in IDS alerts, and it can increase investigation workload.

Despite this advantage, anomaly detection introduces challenges of its own. False positive rates are often higher compared to signature-based systems, increasing the workload for administrators and potentially reducing trust in alerts. Hindy et al. (2018) highlights that anomaly detection models are highly dependent on training data quality, particularly when machine learning techniques are involved. If training datasets are incomplete or biased, detection accuracy may be compromised.

These contrasting strengths and weaknesses demonstrate that IDS effectiveness is not purely a technical issue but also an operational one. Organizations frequently put stability and manageability ahead of theoretical detection superiority in practice. This is particularly important when considering smaller organisations with limited security expertise.

Figure 2 sets out the underlying difference between the two paradigms compared in this study. On the signature-based side, the engine has a list of patterns and an answer ready when one matches; on the protocol-based side, the engine decodes what it sees and writes it down for someone to read later. Both are reasonable approaches, but they answer different questions, and that distinction shapes a lot of the discussion later on.

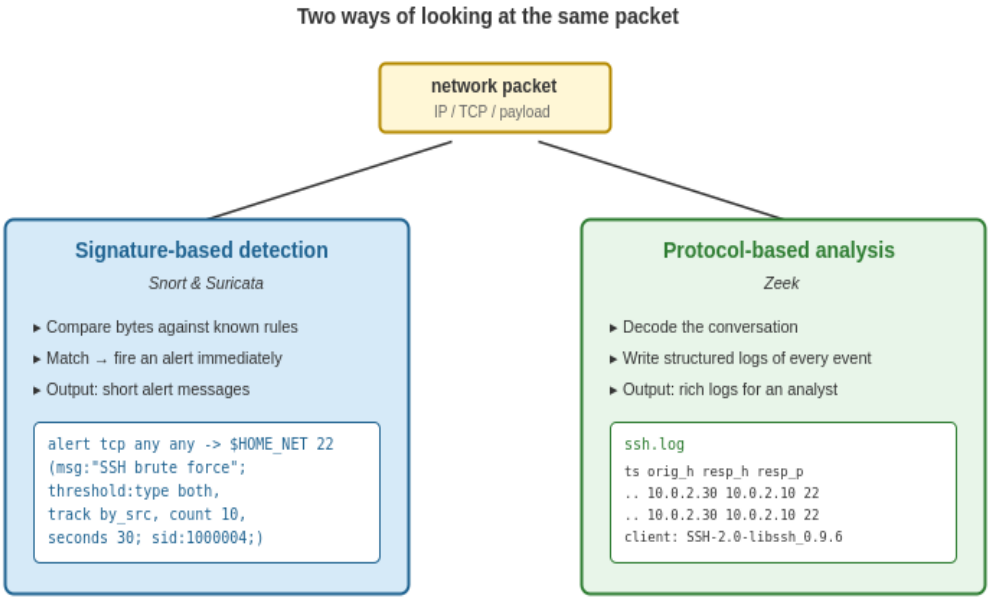

Figure 2. Signature-based detection versus protocol-based analysis – two ways of handling the same packet.

Comparison of Snort, Suricata and Zeek

Because Snort, Suricata, and Zeek are so widely used, comparative studies of open-source intrusion detection systems usually prioritize them. Many studies aim to determine the best performing NIDS system. However, the results are rarely consistent across different research studies and articles.

Snort continues to be regarded as a benchmark for signature-based IDS performance. Ghazi et al. (2024) demonstrate that Snort performs well across several evaluation metrics, especially when analysing structured datasets containing known attack signatures. However, Shah and Issac (2017) report that while Snort achieves high detection rates, it may generate relatively high false positives in certain scenarios. These differing results illustrate how performance metrics can vary depending on dataset characteristics and rule configuration.

Suricata was created partly to overcome Snort's scalability issues. One of its main advantages is simply that it was developed much more recently than Snort, which means it has features built in that are pretty much unmissable these days – multi-threading being the most obvious one. Traffic analysis tasks can be divided among several processor cores due to its multi-threaded architecture. According to Waleed et al. (2022), Suricata outperformed Snort in high-traffic simulations in terms of throughput and packet loss. They also note, though, that detection accuracy improvements aren't always consistent and tend to depend on system tuning and hardware capacity. This suggests that Suricata's advantages may only be realised when sufficient technical expertise and resources are available.

Zeek, on the other hand, places more emphasis on forensic analysis and network visibility instead of direct signature matching. Zeek's ability to generate comprehensive datasets suitable for advanced persistent threat detection and long-term threat analysis is highlighted by Ferriyan et al. (2021). Waleed et al. (2022) also point out that Zeek operates only as an IDS – it doesn't have the inline prevention capability that Snort and Suricata can offer when configured as IPS – and that its richer log output can make it harder to deploy without expert support, especially for businesses without expert cybersecurity teams. As a result, there is a clear trade-off between ease of deployment and analytical depth.

When comparing these different studies, a consistent theme appears. There is no significantly superior IDS tool. Instead, the context heavily impacts the performance outcomes. The generalizability of results is limited since many studies use controlled laboratory datasets instead of real-world network environments. Although laboratory testing increases experimental reliability, Lukaseder et al. (2018) argue that it is unable to accurately replicate the variability of actual organizational networks. As a result, conclusions from benchmark testing should be evaluated with caution.

IDS performance metrics and methodological limitations

The evaluation of IDS systems commonly relies on quantitative performance metrics such as detection accuracy, false positive rate, throughput, and resource consumption. While these metrics are valuable, their interpretation is often influenced by experimental design.

The study released in Computer Networks by Waleed et al. (2022) shows inconsistencies within IDS benchmarking approaches. Different studies utilise unique datasets, traffic simulation techniques, and hardware configurations, making direct comparisons difficult. For example, some research relies on outdated datasets that possibly will not reflect modern cyberattack techniques. Hindy et al. (2018) emphasise that dataset bias remains a major issue

within intrusion detection research, particularly when machine learning techniques are applied.

Similarly, Ahmed et al. (2023) argue that focusing only on detection accuracy can be very misleading. An IDS may demonstrate high accuracy but require effective computational resources, making it impractical for smaller organisations. This reinforces the importance of considering operational efficiency alongside detection effectiveness.

Another limitation involves encrypted traffic. As encryption protocols become standard across internet communications, IDS visibility into packet content is reduced. Al-Dulaimi et al. (2023) suggest that encryption significantly impacts signature-based detection performance, as malicious payloads may be hidden within encrypted streams. Although anomaly detection may offer partial mitigation, it introduces increased processing overhead and higher false positive potential.

These methodological and technological challenges highlight the complexity of IDS evaluation. It becomes clear that performance results should not be interpreted in isolation from deployment context.

Research gap

Looking across all of this, a few gaps stand out. Most of the comparative studies on Snort, Suricata and Zeek run their experiments on enterprise-grade hardware with specialist configuration, which makes the results difficult to map onto smaller organisations. Waleed et al. (2022) points out that there's not much research looking at how these tools behave under the kind of resource limits SMBs typically have. Most existing studies also evaluate each tool in isolation, or only compare two of them at a time, rather than looking at all three under the same conditions. And there's hardly anything in the literature on the practical question of whether running all three at once – which would in theory give better coverage – is even feasible on commodity hardware. This study tries to address those gaps by running a controlled VirtualBox experiment that subjects all three tools to identical attacks on a host that's deliberately limited to SMB-grade resources.

METHODOLOGY

Research design

The study uses a quantitative experimental design carried out in a controlled virtual lab. An experimental approach made sense for the research questions, mainly because the only way to fairly compare three different IDS tools is to put each one through exactly the same conditions, and a virtualised lab makes that level of control achievable. It also keeps the work reproducible – anyone with VirtualBox and the same Ubuntu images can recreate the network, run the same attacks, and compare their findings against the ones reported here.

The work was structured into three experimental phases. Phase one tested each tool on its own – Snort first, then Suricata, then Zeek – against a fixed set of four attacks. Phase two looked at how the detection techniques themselves compared. Snort and Suricata both use signature-based detection, but they aren't the same engine, and the same traffic produced different alert counts and different alert categories on each. Zeek doesn't really produce alerts in the same sense, so it had to be analysed differently, mostly through its log files. Phase three was a separate resource test where more than one tool ran at the same time during an active attack, to see whether SMB-grade hardware could actually handle that kind of stack.

Why small businesses can't just run all three

It's a fair question to ask, before doing any of this, why an organisation wouldn't just install all three tools and let them all run – on paper, that gives the most coverage. In practice there are a few reasons that doesn't really work for an SMB.

The first is hardware. Each of these tools eats a noticeable amount of CPU and memory while it's processing live traffic. Waleed et al. (2022) found that even one tool on its own can saturate a modest server when traffic volume goes up. Running three of them at once multiplies that demand, and the kind of commodity hardware most small businesses can afford simply doesn't have the headroom.

The second is alert management. Snort and Suricata produce streams of alerts in different formats and at different volumes; Zeek doesn't produce alerts at all, just logs. Pulling those three sources together meaningfully needs either a SIEM, which is its own cost and operational headache, or a person who knows what they're doing across all three frameworks. Most SMBs have neither. Ahmed et al. (2023) make the point that the value of an IDS depends on whether the organisation can actually act on what it produces – a tool that drowns a generalist IT admin in alerts they can't triage is arguably worse than no tool at all.

The third is expertise. Snort wants careful rule selection, Suricata needs ongoing rule and threading tuning, Zeek needs someone who can read its log schema and ideally write its scripts. Anyone who can run all three well is, more or less by definition, a security specialist – and almost no small business employs one full time. The fourth is the cost of false positives stacking up. Garcia-Teodoro et al. (2009) documented alert fatigue as a serious problem, and the same idea applies when several tools' worth of false positives all land on the same person's desk. The result, in practice, is that the alerts get ignored and the supposed extra coverage delivers nothing.

Together, these reasons justify the focus of this study on identifying the best single tool – or workable combination of two – for an SMB context, rather than assuming the three-tool stack is realistic.

Test environment

The lab was built on a Windows host running VirtualBox 7. Three virtual machines were created and connected through a single isolated NAT network in the 10.0.2.0/24 range, named IDS_Lab. Figure 1 shows the topology and the role of each machine.

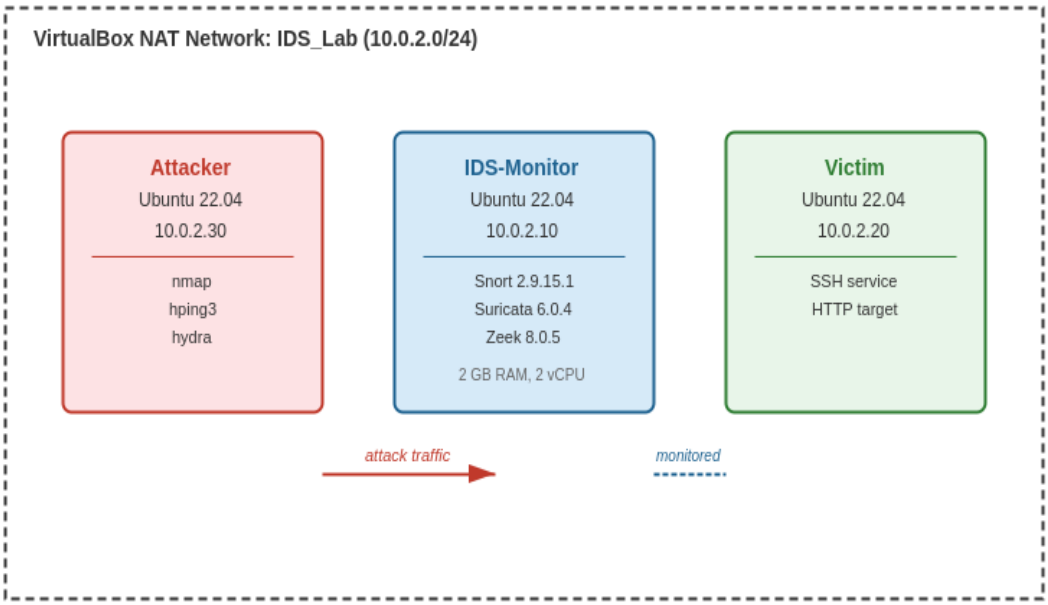

Figure 1. Virtual network topology used for the experiments. All three machines share the IDS_Lab subnet.

The hardware allocation for each machine is shown in Table 1. The IDS-Monitor was deliberately kept on the small side – 2 GB of RAM and two CPU cores – so that it would resemble the kind of low-end server an SMB might allocate to a monitoring role. The Attacker and Victim were given even less, since they only needed to generate or accept traffic.

Machine	OS	RAM	CPU	Disk	IP address
IDS-Monitor	Ubuntu Server 22.04	2 GB	2 vCPU	20 GB	10.0.2.10
Victim	Ubuntu Server 22.04	1 GB	1 vCPU	10 GB	10.0.2.20
Attacker	Ubuntu 22.04	1 GB	1 vCPU	10 GB	10.0.2.30

Table 1. Hardware allocation for each virtual machine.

Each VM had two network adapters: an internal NAT Network adapter so the three machines could speak to each other on 10.0.2.0/24, and a standard NAT adapter for outbound internet during installation and updates. Port forwarding rules on the host let SSH traffic into the VMs from the host's Windows command line, which made it a lot easier to copy commands between machines during the experiments.

Tools installed on the IDS-Monitor

All three IDS tools were installed on the IDS-Monitor. Snort 2.9.15.1 came from the Ubuntu universe repository. Suricata 6.0.4 came from the same source and was updated using suricata-update. Zeek 8.0.5 had to be installed from the openSUSE Build Service repository because it isn't packaged in the standard Ubuntu archive. The exact versions are listed in Table 2.

Tool	Version	Source
Snort	2.9.15.1	Ubuntu 22.04 universe repository
Suricata	6.0.4	Ubuntu 22.04 universe repository, plus suricata-update for ET Open rules
Zeek	8.0.5	openSUSE Build Service repository (download.opensuse.org)

Table 2. Tool versions and where they were installed from.

On the Attacker VM, three tools were used to generate the test traffic: Nmap 7.80 for the port scan, hping3 for both flood attacks, and Hydra 9.2 for the SSH brute force. All three are standard penetration-testing utilities available from the Ubuntu archive.

Attacks used

Four attack types were chosen because they're common, easy to reproduce, and stress different parts of an IDS. A port scan is essentially reconnaissance and tests whether the IDS can spot an unusual pattern of half-open TCP connections. An ICMP flood is a very high-volume noise attack at the network layer. A SYN flood is a similarly noisy denial-of-service attempt aimed at the TCP layer. An SSH brute force is the most application-layer of the four – it requires the IDS to either have a dedicated SSH rule, or log the protocol in enough detail to spot the pattern after the fact. The exact commands used are in Table 3.

Attack	Command on Attacker	Run time
Port scan	sudo nmap -sS 10.0.2.10	≈ 30 seconds
ICMP flood	sudo hping3 -1 --flood 10.0.2.10	30 seconds
SYN flood	sudo hping3 -S --flood -p 80 10.0.2.10	30 seconds
SSH brute force	hydra -l idsuser -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.10 -t 4	2–3 minutes

Table 3. The four attack scenarios are used in every experiment.

Data collected

For each attack and each tool, the following information was recorded:

- The number of alerts produced (Snort and Suricata) or log entries written (Zeek).
- The category and message text of each alert, where applicable.
- Peak CPU usage during the attack, read from htop on a second SSH session into the IDS-Monitor.
- Peak RAM usage, also from htop.
- Whether the IDS service or the host showed any signs of stress – becoming unresponsive, being killed by the OS, that kind of thing.

All four attacks were run in the same order each time, with a short pause between them to let the host settle. Snort was tested first, then Suricata, then Zeek; the resource test that combined more than one tool came last.

Ethical considerations

All of the experimental work was done inside a self-contained virtual lab. The NAT Network in VirtualBox was configured so that traffic from the Attacker VM couldn't reach the host machine, the host's home network, or anywhere on the public internet. No real organisation, no real user data, and no live system was touched at any point. The only credentials used were the ones I created myself for the lab, and the only target was a VM running on my own laptop. The project proposal was reviewed and approved by my supervisor in line with Sheffield Hallam University's standard procedures, and no extra ethical approval was needed because the work involved no human participants and no live infrastructure outside the lab.

Limitations

Two limitations are worth flagging up front, because they shape how the results in Section 4 should be read. First, the lab is a virtual one. As Lukaseder et al. (2018) pointed out, virtual labs can never quite reproduce the variability of a real organisation's network, and the traffic in this study is the traffic I generated on purpose – which is much cleaner than what an IDS in production would actually see. Second, the resource cap of 2 GB of RAM was deliberately chosen to mirror typical SMB hardware, but it is at the bottom end of what these tools really want; some of the results, especially the Suricata default-rule failure, probably wouldn't have happened on a 4 GB machine. Both limitations were accepted because they make the study more relevant to the SMB context, not less – if a tool needs more than 2 GB just to start up, that's itself a useful finding for an SMB.

RESULTS

Overview

This section presents the results from each of the three experiments and the resource test. Each tool was run against the same four attacks: an Nmap port scan, an ICMP flood, a SYN flood and an SSH brute force. Where Snort and Suricata produced real-time alerts, those were counted and categorised. Where Zeek produced log files instead, the relevant logs were examined to see what the tool had actually captured. Resource usage (CPU and RAM) was recorded for each attack from a second SSH session running htop. Figure 3 shows the headline finding before the detail is broken out tool by tool.

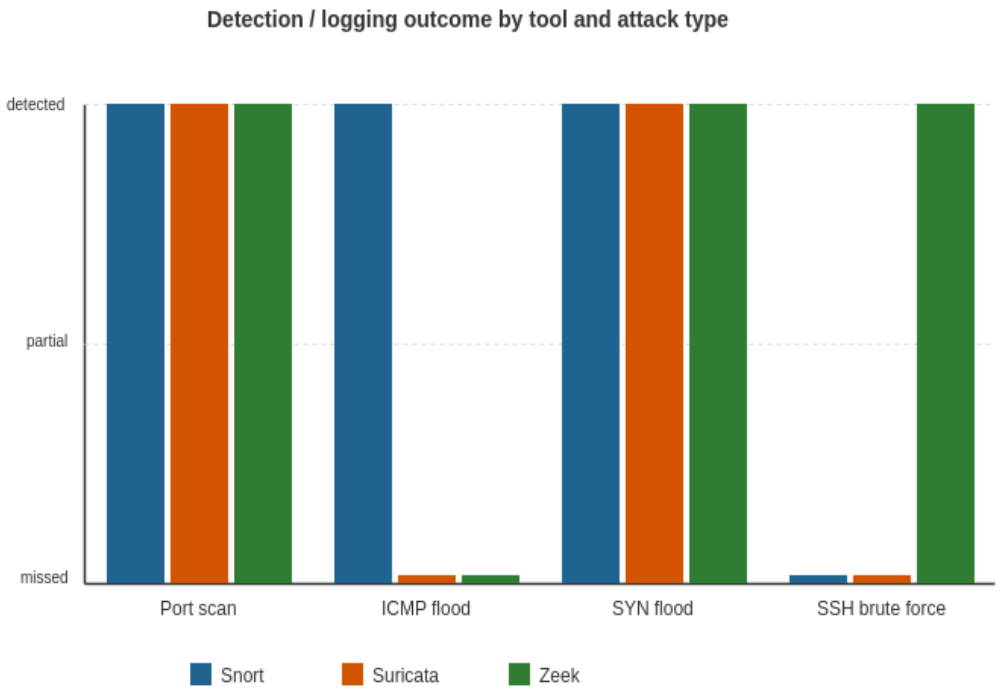

Figure 3. Overall detection / logging outcome for each tool against each attack.

Experiment 1: Snort

Snort was straightforward to install and run. After validating the configuration with snort -T the engine started cleanly with sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8 and started printing alerts as soon as traffic appeared on the IDS_Lab network. No tuning was needed beyond setting the HOME_NET variable in snort.conf to 10.0.2.0/24.

Port scan

Within seconds of the Nmap scan starting, Snort produced its first alerts. Two distinct rules fired – a SNMP request tcp alert and a SNMP AgentX/tcp request alert, both classified as Attempted Information Leak with priority 2. The alerts correctly identified 10.0.2.30 as the source and 10.0.2.10 as the destination. The Nmap scan itself only found port 22 (SSH) open on the target.

ICMP flood

During the 30-second ICMP flood, hping3 sent 2,569,125 packets at the IDS-Monitor. Snort produced ICMP PING NMAP alerts (again Attempted Information Leak priority 2) at a rate

of hundreds per second, scrolling too fast to read in the terminal but clearly firing. Peak CPU during the flood reached around 94% on Core 0, with the second core also significantly loaded.

SYN flood

The SYN flood was the most varied attack from a detection point of view. hping3 transmitted 2,173,186 packets in 30 seconds and Snort produced four distinct alert types in response: BAD-TRAFFIC tcp port 0 (priority 3), MISC Source Port 20 to <1024 (priority 2), MISC source port 53 to <1024 (priority 2), and SNMP trap tcp (priority 2). That breadth reflects how Snort's default community rules try to flag anything unusual at the TCP layer, not just the flood itself. CPU peaked around 94% on the first core, and RAM rose noticeably, from a baseline of about 984 MB to 1.04 GB during the flood – probably because Snort's connection table was filling up with half-open SYNs.

SSH brute force

The SSH brute force was run with hydra -l idsuser -P /usr/share/wordlists/rockyou.txt ssh://10.0.2.10 -t 4. Hydra found the credentials in 13 seconds after seven attempts. Snort, however, didn't generate any alerts at all in response. This lines up with Shah and Issac (2017), who noted that Snort's default rule sets don't really contain SSH-specific brute-force detection logic. It's also a fairly clean example of the application-aware limitation discussed in the literature review – Snort sees SSH traffic as a stream of TCP packets going to port 22, but without rules that understand what an SSH login attempt looks like, it has no way to recognise the pattern of repeated failed authentications.

Snort summary

Attack	Detected?	Alert categories	Peak CPU / RAM
Port scan	Yes	SNMP request tcp; SNMP AgentX/tcp request	≈ 5% / 1.14 GB
ICMP flood	Yes	ICMP PING NMAP (hundreds/sec)	≈ 94% / 0.98 GB
SYN flood	Yes	BAD-TRAFFIC tcp port 0; MISC Source Port 20; MISC source port 53; SNMP trap tcp	≈ 94% / 1.04 GB
SSH brute force	No	(no alerts produced under default rules)	no spike

Table 4. Snort detection summary.

Experiment 2: Suricata

Suricata took quite a bit more work to get running than Snort, and the reason was memory. After running suricata-update, the rule set on the IDS-Monitor contained 49,828 enabled rules out of 65,703 total. Trying to validate the configuration with sudo suricata -T -c /etc/suricata/suricata.yaml caused the process to be terminated by the OS – the only output was the single word "Killed" before the prompt came back. The load of holding all 49,828 rules in memory plus Suricata's flow tables was simply too much for a 2 GB VM.

That on its own is a useful finding. Waleed et al. (2022) noted that Suricata's performance benefits depend on adequate hardware, and this experiment puts a fairly concrete number on what "adequate" means: Suricata couldn't even pass its own configuration test on 2 GB of RAM with the default rule set. To make the experiment go anywhere, the rule set was replaced with a small custom file containing four targeted rules – one each for ICMP flood, SYN flood, port scan and SSH brute force – using Suricata's threshold and detection_filter keywords. After that change Suricata started cleanly.

Port scan

Suricata flagged the Nmap scan with the custom Port Scan Detected rule (SID 1000003) within seconds. fast.log filled up with entries showing 10.0.2.30 contacting the IDS-Monitor on a wide range of TCP ports.

ICMP flood

During the 30-second ICMP flood, fast.log produced no entries at all. The custom rule had a threshold of 100 ICMP type-8 packets per second, which the flood comfortably exceeded, but no alerts came through. The most likely cause is a syntax issue in the threshold parameters of the custom rule – but it does show how easy it is to misconfigure Suricata in a way that silently breaks detection.

SYN flood

The SYN flood was the most cleanly detected of all the Suricata results. Both the SYN Flood Detected (SID 1000002) and Port Scan Detected (SID 1000003) rules fired continuously while the flood ran, with timestamps essentially indistinguishable from each other. Peak CPU during the flood reached about 71% – lower than Snort's because Suricata distributed work across both vCPUs.

SSH brute force

Suricata didn't produce any alerts during the SSH brute force test. The custom SSH rule used the content keyword to look for the SSH- protocol banner, combined with a threshold of 10 connections in 30 seconds. The pattern was almost certainly being matched at the TCP layer, but no alert ended up in fast.log. As with the ICMP rule, the most likely explanation is rule syntax – SSH brute force detection in Suricata generally needs proper application-layer parsing rather than raw content matching. The result for the SMB context is the same as Snort's: SSH brute force not detected.

Suricata summary

Attack	Detected?	Alert categories	Peak CPU / RAM
Port scan	Yes	Port Scan Detected (SID 1000003)	≈ 30% / 1.10 GB
ICMP flood	No	(rule present but did not fire)	≈ 60% / 1.10 GB
SYN flood	Yes	SYN Flood Detected (SID 1000002); Port Scan Detected (SID 1000003)	≈ 71% / 1.20 GB
SSH brute force	No	(custom rule did not fire)	no spike

Table 5. Suricata detection summary, after the rule set was reduced from 49,828 down to four custom rules.

Experiment 3: Zeek

Zeek behaves quite differently from the two signature-based engines, and the results have to be read in a different way. It doesn't produce alerts in real time – it produces structured logs. Each attack therefore had to be examined by reading the relevant Zeek log files after the traffic had stopped, rather than watching alerts scroll past during the attack itself.

After installing Zeek and configuring node.cfg with interface=enp0s8, the engine was started with Sudo /opt/Zeek/bin/zeekctl deploy and confirmed running with zeekctl status. Zeek runs as the root user and writes logs to /opt/Zeek/logs/current/, so the analysis commands all needed Sudo.

Port scan

During the Nmap scan, Zeek produced a stream of entries in conn.log. Each entry showed 10.0.2.30 connecting to a different TCP port on 10.0.2.10, almost all of which were marked with the conn_state "REJ" (rejected). That rejection state is the connection-layer fingerprint of a SYN scan against closed ports – each of the SYNs Nmap sent received an RST in reply because no service was listening. The signature is unmistakable in the log even though there's no explicit "port scan" alert.

ICMP flood

Zeek didn't produce a separate icmp.log under the default configuration on this system, and conn.log captured only a small number of entries during the flood. The flood traffic was high enough to overwhelm Zeek's connection tracking at default settings; on a more capable host with the Zeek icmp protocol analyser explicitly enabled, the result would probably be different. On the SMB-grade hardware used here, though, the practical outcome was that Zeek captured no clear evidence of the ICMP flood.

SYN flood

This was the most informative attack from a Zeek standpoint. conn.log produced thousands of entries, each showing a single SYN packet from 10.0.2.30 to port 80 on 10.0.2.10, all marked with conn_state "REJ". The duration of every connection was measured in microseconds, and the byte counts on both sides were zero, because the connections never completed. Zeek didn't raise an alert, but the evidence in conn.log is more than enough for an analyst to recognise the attack after the fact – a single source generating tens of thousands of failed half-open connections to the same destination port in a few seconds is unambiguously a SYN flood.

SSH brute force

This is the result that separates Zeek most sharply from the other two tools. Zeek wrote each SSH connection attempt to ssh.log, including the source and destination IPs, source and destination ports, and – most usefully – the SSH client version string, which on Hydra-generated traffic shows up as "SSH-2.0-libssh_0.9.6". That client banner is a strong indicator that the connections came from an automated tool rather than a normal SSH client like OpenSSH. Combined with the volume of attempts and the fact that the AUTH success column shows no successful logins, ssh.log gives exactly the kind of forensic evidence that would let an analyst conclude, after the fact, that an SSH brute force had taken place. Snort and Suricata both missed this attack entirely.

There was one important caveat. The first attempt at this experiment failed because the SSH service on the IDS-Monitor stopped responding while the experiment was in progress. htop showed Snort still running in the background from the previous experiment, and the combined load of Snort, Zeek and the SYN flood traffic from earlier had pushed the host into resource contention. After Snort was stopped and SSH restarted, the brute force completed normally and Zeek captured the attempts as described. The incident is reported in detail in the resource test below.

Zeek summary

Attack	Logged?	Where the evidence appears	Peak CPU / RAM
Port scan	Yes	conn.log – many REJ states from one source	≈ 8% / 0.45 GB
ICMP flood	No	(default config did not retain a useful icmp record)	≈ 18% / 0.45 GB
SYN flood	Yes	conn.log – thousands of REJ states to port 80	≈ 20% / 0.50 GB
SSH brute force	Yes	ssh.log – every attempt, with libssh client banner	≈ 5% / 0.45 GB

Table 6. Zeek logging summary.

Experiment 4: Concurrent resource test

The last experiment looked at what happens when more than one IDS tool runs on the same host during an active attack. It was triggered by the unplanned observation during the Zeek experiment described above – with Snort still running in the background, the IDS-Monitor became unresponsive enough that Hydra couldn't even open a TCP connection to the SSH service. The htop readings at that moment showed Snort using 53% CPU and Zeek using 3.2% CPU, with combined RAM use at 1.6 GB out of the 1.92 GB available – about 83% of the VM's total memory. Both CPUs were saturating somewhere between 78% and 100% during the SYN flood that was still running at the same time. Once Snort was killed and the SSH service restarted, the brute force completed and Zeek captured it as described in the previous section. Figure 4 puts the resource numbers into context.

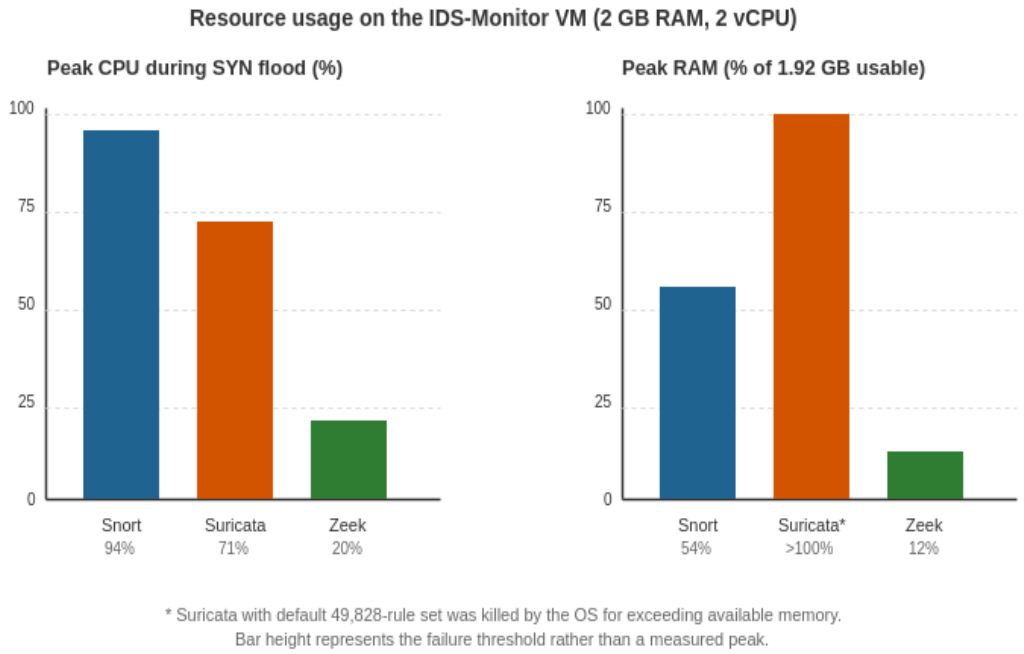

Figure 4. Peak CPU and RAM during a SYN flood for each tool running on its own. The Suricata RAM bar represents the failure threshold rather than a measured peak, because Suricata with the default rule set could not start at all.

Two readings to take from this. First, no single tool on its own came particularly close to using all the host's resources during a flood – Snort topped out at about 94% CPU and just over half of available RAM, Zeek used very little of either. Second, the combination is a different story: the moment Snort and Zeek were running at the same time during a flood, the host became unable to provide its core SSH service. For an SMB, that's not a theoretical problem – it's a real outage caused by the security tools themselves.

Cross-tool comparison

Pulling the three tools together, Table 7 summarises the headline outcome for each attack. It also makes the central point of the project visible at a glance: no one tool wins everywhere, and the disagreements aren't random – they line up with the underlying detection paradigm.

Attack	Snort	Suricata	Zeek
Port scan	Detected (SNMP rules)	Detected (custom rule)	Captured in conn.log
ICMP flood	Detected (PING NMAP)	Missed (rule did not fire)	Not captured cleanly
SYN flood	Detected (4 rule types)	Detected (custom rules)	Captured in conn.log
SSH brute force	Missed	Missed	Captured in ssh.log

Table 7. Combined outcome across the three tools.

DISCUSSION

Answering the research questions

The headline pattern in Table 7 is the cleanest place to start. Each research question can now be addressed against experimental evidence rather than just literature.

RQ1 – Which tool is most reliable on 2 GB of RAM with no expert tuning?

The clearest answer here is Snort. It installed cleanly, ran without complaint on its default rule set, and detected three of the four attack types without any tuning at all. Suricata couldn't even pass its own configuration test under default rules on the same hardware and only ran after manual rule reduction. Zeek ran fine and used barely any resources but produced no real-time alerts at all – which means an SMB admin watching the host during an attack would see nothing happening unless they knew which log files to read. On the specific question of out-of-the-box reliability with no tuning, Snort is the only one that really meets the bar.

RQ2 – what configuration barriers does each tool present?

Snort presented essentially no barrier beyond setting the HOME_NET variable. Suricata presented the most serious barrier of any of the three: its memory consumption with the default rule set caused the process to be killed outright on a 2 GB VM, and recovering from that needed either upgrading the hardware or hand-pruning the rule set – neither of which is trivial for someone who isn't already a security specialist. Zeek presented a moderate barrier of a different sort. The installation needed an extra third-party repository, and the system has to be configured through zeekctl rather than the more familiar systemctl, but once it was running it stayed running. The bigger barrier with Zeek is conceptual really – an admin used to see alerts will find Zeek silent and must learn that the value lives in the logs.

RQ3 – Can two tools run together on this hardware during an attack?

Based on the resource test, no, not safe. Running Snort and Zeek concurrently during a SYN flood pushed the host's RAM use to 83% and CPU to 78–100% on both cores, and the SSH service stopped responding. It's worth being honest about what this test does and doesn't show – it's one observation, on one set of VM resources, with one combination of tools and one attack. But the direction of the result is unambiguous: combining two of these tools on a 2 GB host during an active attack creates resource contention severe enough to take a service offline. For an SMB whose IT person also relies on SSH being up to do their job, that's a meaningful finding.

RQ4 – What trade-offs exist between signature-based detection and Zeek's protocol-based logging?

This is the question the project was really designed to answer, and Figure 2 in the literature review is the cleanest way to think about it. Signature-based detection answers the question "is anything happening right now that matches a known bad pattern?" and answers it loudly the moment a match occurs. Protocol-based analysis answers the question "what happened on the network, in detail?", and answers it later, in writing. Neither is wrong – they're good at different things.

The SSH brute force is a textbook example. Snort and Suricata both missed it; the default Snort rule set has nothing useful for SSH brute force, and Suricata's rule syntax for application-layer detection is more involved than a simple custom rule could meet. Zeek

captured every single attempt, with enough metadata to identify the actual attack tool used. But it did all of this only after the fact – at no point during the attack did the IDS-Monitor signal that anything was wrong. The SYN flood works the other way around: both signature-based tools alerted on it within milliseconds, with several rule categories firing simultaneously, which would let an SMB take action immediately. Zeek captured it too, but its evidence is in conn.log and only really makes sense when read in aggregate.

Comparison with the literature

The findings line up with the existing literature in some ways and push back on it in others. Waleed et al. (2022) found Suricata to be the highest-throughput tool of the three when given enough hardware. This study can't really dispute that, but it adds a fairly pointed caveat – at 2 GB of RAM the throughput advantage is irrelevant, because Suricata can't start with its default rule set at all. Hardware adequacy isn't an ancillary detail for SMBs; it's the gating condition. Similarly, Shah and Issac (2017) reported that Snort can produce significant false positives at high traffic rates. The behaviour observed in this study during the SYN flood – four different rule categories firing, including SNMP rules that have nothing to do with the actual attack – is consistent with that finding. Snort's default rule set is broad rather than precise, and the result is alert volume that an SMB without an analyst would struggle to triage.

Ferriyan et al. (2021) and other authors writing about Zeek emphasise its forensic depth. The ssh.log evidence captured here is a small but concrete demonstration of that strength: every connection attempt was recorded with enough context to identify the tool used. However, Waleed et al. (2022) also noted that Zeek lacks inline prevention capability, and that observation matters here too. Zeek told the experiment, in writing, that an SSH brute force had happened. It didn't – and couldn't – stop the brute force from succeeding. An SMB that picks Zeek alone is choosing visibility over enforcement.

A point this study can make more directly than most of the literature is the operational consequence of trying to run more than one tool on small hardware. Ahmed et al. (2023) raised the principle that an IDS's value depends on the organisation's capacity to act on its output. The resource test here adds a related principle, which is that an IDS's value also depends on the host being able to keep running while the IDS is running. Saturating a 2 GB host with two tools and an attack isn't a hypothetical – it actually happened during the SSH experiment and took a service offline.

Some caveats

Some honest qualifications are worth recording. The Suricata ICMP and SSH rules used in the experiment didn't fire even when the underlying traffic should have matched. On a different system, or with more carefully written rules, both attacks could probably have been caught. What that means isn't that Suricata is incapable of detecting these attacks – the literature is clear that it can – but that catching them requires expertise an SMB admin is unlikely to have. From an SMB-deployment standpoint, the practical result is the one that matters: with reasonable effort and reasonable rule writing, Suricata caught two of four attacks on this hardware, and that's the data point an SMB needs.

Likewise, Zeek's missed ICMP flood isn't really evidence that Zeek is bad at handling ICMP. It's evidence that Zeek's default packet-handling on a 2 GB VM can't keep up with a 100,000+

packet-per-second flood. On a host with more resources, or with the icmp protocol analyser explicitly tuned, the result would almost certainly be different. Again, the SMB interpretation is the one that matters: Zeek's default behaviour on small hardware doesn't give SMB clear evidence of an ICMP flood, and an SMB admin shouldn't assume it would.

CONCLUSION

Summary of the findings

The aim of this project was to compare Snort, Suricata and Zeek when deployed on the kind of low-spec hardware that small businesses can realistically afford, and to translate that comparison into something useful rather than into another set of throughput numbers. The four experiments and the resource test produced a consistent picture, which is worth restating in plain terms.

Snort installed in seconds, ran on default rules, and detected the port scan, the ICMP flood and the SYN flood. It missed the SSH brute force completely. It used the most CPU of the three tools but was the easiest to deploy. Suricata couldn't run on default rules at all on a 2 GB VM – the OS killed the configuration test for using too much memory. After manual rule reduction, it detected the port scan and the SYN flood but missed the other two. Zeek ran cleanly, used the least resources of the three, and produced no real-time alerts but captured the SSH brute force in detail in ssh.log. Running Snort and Zeek at the same time during an active flood pushed the IDS-Monitor into resource contention severe enough to take its SSH service offline.

Process and limitations

Some honest reflection on the process is worth recording. The experiments were carried out in a virtualised lab on a single laptop, using a deliberately small host VM. That choice was the right one for the research questions, but it does mean some of the failures observed – particularly Suricata's default-rules failure and Zeek's missed ICMP flood – are partly a product of the small hardware envelope and probably wouldn't reproduce on, say, a 4 GB host. The custom Suricata rules used in the experiment also weren't professionally written. An experienced security engineer could probably have got Suricata to detect all four attacks; what the experiment shows is that an SMB admin working from default tooling and reasonable effort just won't.

Two improvements would strengthen any follow-up study. First, repeating the experiments on hardware envelopes of 2 GB, 4 GB and 8 GB would draw a meaningful curve of where each tool actually becomes practical. Second, including encrypted traffic – TLS-protected SSH brute force, for example, or HTTPS-based scans – would test how each tool copes with the encrypted reality of modern networks, an area that Al-Dulaimi et al. (2023) flagged as a serious open problem.

Recommendations

From the evidence in this study, the practical advice for a small business with around 2 GB of RAM and no security specialist comes in three parts.

Pick Snort first. It's the tool that most reliably runs out of the box on this hardware and produces the fastest, loudest alerts when something obvious is happening. Accept that it'll

miss credential-based attacks like SSH brute force and compensate by hardening the SSH service itself – disable password logins, enforce key authentication, install fail2ban for failed-attempt limiting.

Add Zeek second, but not on the same host. Zeek is genuinely good at the kind of evidence Snort doesn't produce, and the SSH brute force result here makes that point cleanly. But the resource test shows that running it alongside Snort on a 2 GB host can take services offline during attacks. If Zeek is going to be added, it should go on a separate host – even if that host is also small.

Avoid Suricata unless the SMB has access to security expertise or can budget for at least 4 GB of RAM. It isn't that Suricata is a bad tool – the literature is pretty clear that it's fast and capable – but its memory footprint and rule-management complexity put it out of reach for the SMB profile this study targets. An SMB that hires a security consultant for a day or two could probably get Suricata running well, but in the absence of that consultant, Snort is the safer bet.

More broadly, the study seems to support a principle that's implicit in much of the literature but rarely stated outright: the right IDS for an SMB isn't the most powerful one, but the one its administrator can actually keep running. Hardware adequacy, configuration simplicity and operational manageability aren't secondary considerations in the SMB context; they're the considerations.

REFERENCES

Ahmed, M., Naser Mahmood, A. and Hu, J. (2016) 'A survey of network anomaly detection techniques', Journal of Network and Computer Applications, 60, pp. 19–31. https://doi.org/10.1016/j.jnca.2015.11.016

Al-Dulaimi, K., Al-Sabahi, M. and Venkatesh, S. (2023) 'Impact of encryption on network intrusion detection systems: a systematic review', IET Information Security, 17(4), pp. 590–610. https://doi.org/10.1049/ise2.12120

Ferriyan, A., Thamrin, A. H., Takeda, K. and Murai, J. (2021) 'Generating network intrusion detection dataset based on real and encrypted synthetic attack traffic', Applied Sciences, 11(17), p. 7868. https://doi.org/10.3390/app11177868

Garcia-Teodoro, P., Diaz-Verdejo, J., Maciá-Fernández, G. and Vázquez, E. (2009) 'Anomaly-based network intrusion detection: techniques, systems and challenges', Computers and Security, 28(1–2), pp. 18–28. https://doi.org/10.1016/j.cose.2008.08.003

Ghazi, D. S., Shehab, H., Zaiter, M. J., Sabri, A. and Behadili, G. (2024) 'Snort versus Suricata in intrusion detection', Iraqi Journal of Information and Communication Technology, 7(2), pp. 73–88. https://doi.org/10.31987/ijict.7.2.290

Hindy, H., Brosset, D., Bayne, E., Seeam, A., Tachtatzis, C., Atkinson, R. and Bellekens, X. (2018) 'A taxonomy and survey of intrusion detection system design techniques, network threats and datasets', arXiv preprint, arXiv:1806.03517. https://arxiv.org/abs/1806.03517

Lukaseder, T., Kleber, S. and Kargl, F. (2018) 'A comparison of network traffic generators', in Proceedings of the 33rd Annual ACM Symposium on Applied Computing. New York: ACM, pp. 1670–1677. https://doi.org/10.1145/3167132.3167185

Paxson, V. (1999) 'Bro: a system for detecting network intruders in real-time', Computer Networks, 31(23–24), pp. 2435–2463. https://doi.org/10.1016/S1389-1286(99)00112-7

Roesch, M. (1999) 'Snort — lightweight intrusion detection for networks', in Proceedings of the 13th USENIX Systems Administration Conference (LISA '99). Seattle: USENIX, pp. 229–238.

Shah, S. A. R. and Issac, B. (2017) 'Performance comparison of intrusion detection systems and application of machine learning to Snort system', Future Generation Computer Systems, 80, pp. 157–170. https://doi.org/10.1016/j.future.2017.10.016

Sommer, R. and Paxson, V. (2010) 'Outside the closed world: on using machine learning for network intrusion detection', in Proceedings of the 2010 IEEE Symposium on Security and Privacy. Oakland: IEEE, pp. 305–316. https://doi.org/10.1109/SP.2010.25

Waleed, A., Jamali, A. F. and Masood, A. (2022) 'Which open-source IDS? Snort, Suricata or Zeek', Computer Networks, 213, p. 109116. https://doi.org/10.1016/j.comnet.2022.109116